HopIn

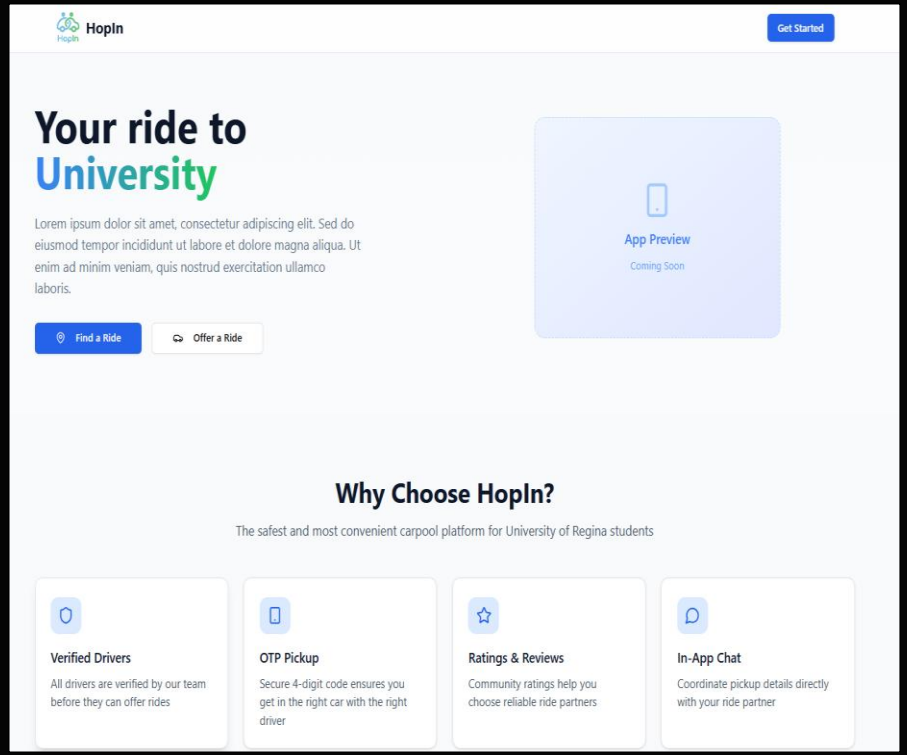
University Carpooling Platform

Jessica, Rida, Rudra, Suhayb, Youssef

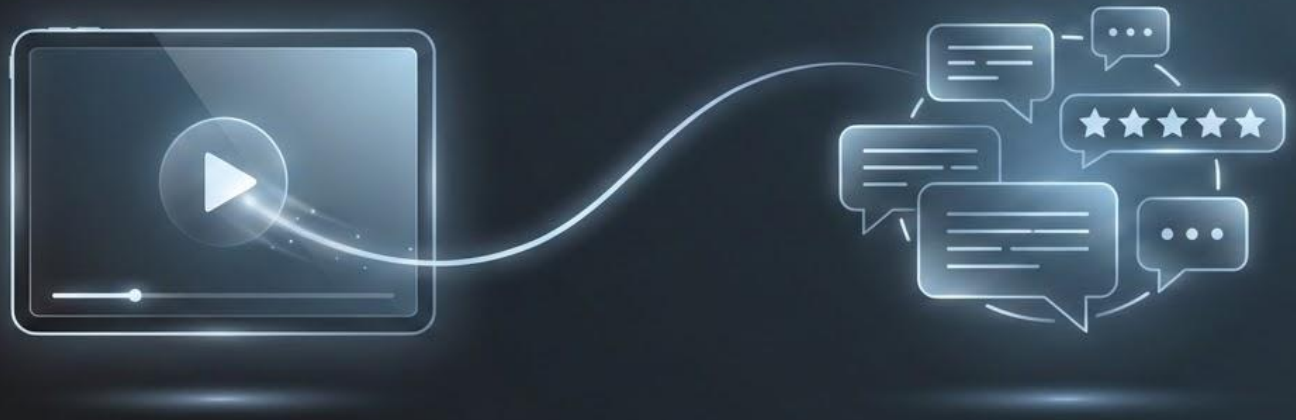
Why, How, Who, What

Commuting to university can be a challenge due to rising transportation costs, limited public transit, and car issues, especially in the winter. HopIn addresses these challenges by connecting drivers with available seats to riders seeking reliable and safe transportation.

With the university student in mind, HopIn gives users a smooth experience finding people in their area to either offer a ride, or get a ride to the university.



Demo + Feedback



Project Planning Methods

Ride Posting Feature	Drivers will be able to create ride listings including details such as departure location, destination, date, time, and number of available seats.
Ride Search and Discovery	Riders will be able to browse and search for rides based on destination, date, and availability.
Ride Request Management	Riders will be able to request seats in available rides. Drivers will have the ability to accept or decline these requests.
Trip Management Dashboard	Users will be able to view rides they have posted, requested, or confirmed through a personal dashboard.
Messaging Feature	Drivers and riders can communicate necessary trip details such as meeting locations or time adjustments.
Database System	The system will include a backend database to store user accounts, ride listings, and ride requests.
Web-Based User Interface	The project will include a functional web interface that allows users to interact with the platform and perform all required features.

- Decided to focus on 7 core features
- Excluded:
  - Integration with external payment system
  - Liability management
  - 24/7 customer support

Project Scope Statement:
Project Deliverables

Project Planning Methods

- First we established the requirements and the required event handling
- Establishing parameters
- Designed the front end and back end architectures
- Focused on models, followed by coding

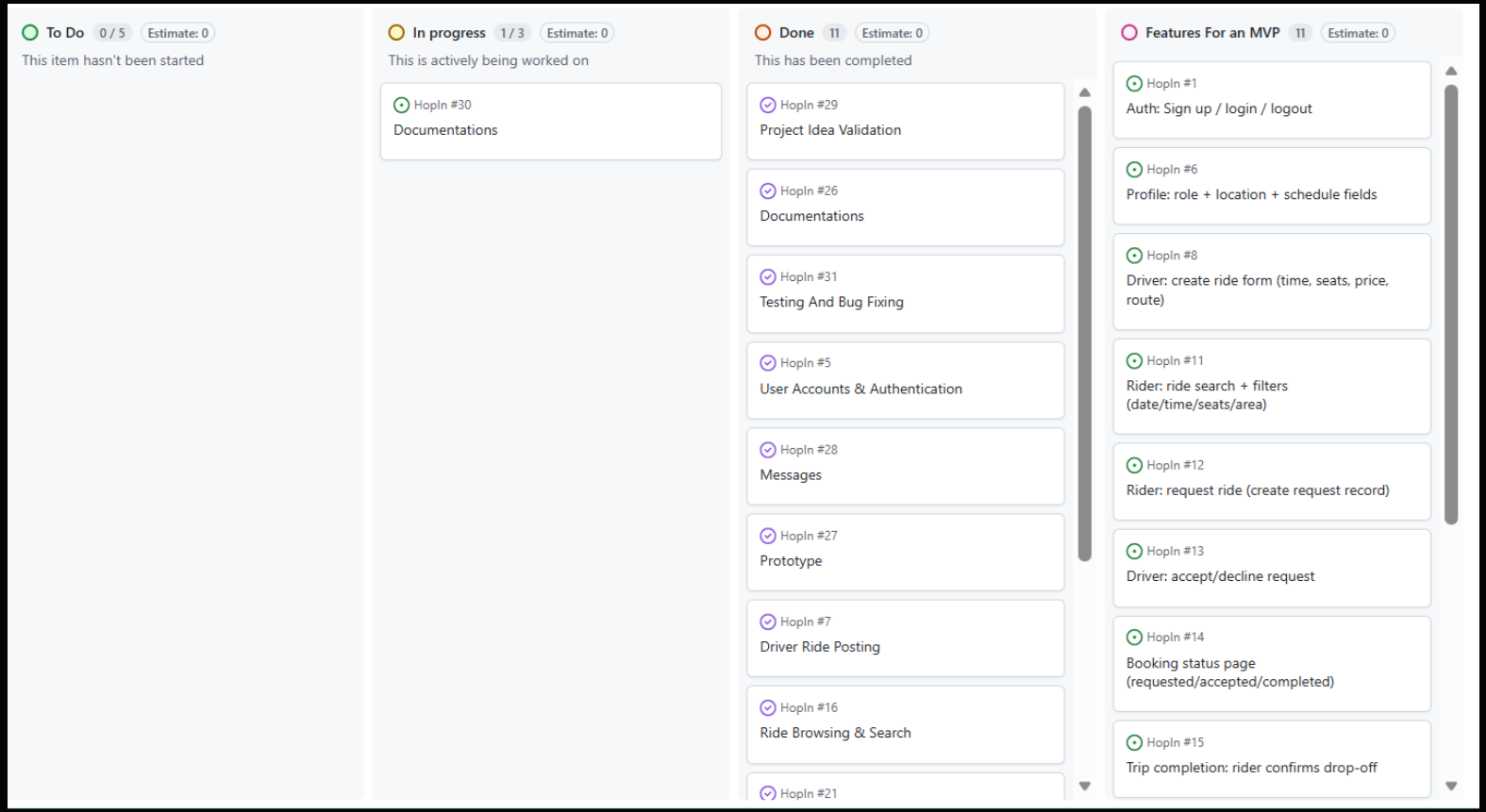
Functional Requirements

- User account management:
  - **allows users to create account as a rider and opt in as a driver**
  - **users can log in and log out securely**
  - **update account information (name, phone number, email, password)**
- Schedule and Commute Management:
  - **Allow users to input their class schedule and/or commute times**
  - **Edit or delete schedule entries**
  - **store and display upcoming commute times for both drivers and riders**
- Ride and Search Matching:
  - **System will allow all riders to search for available drivers based on:**
    - **Time of commute**
    - **Pickup area or neighbourhood**
    - **Driver availability**
  - **Display list of matching drivers with relevant details (name, vehicle, time, rating**
  - **Allow user to message driver within the app to plan the ride**

Project Requirements Document:

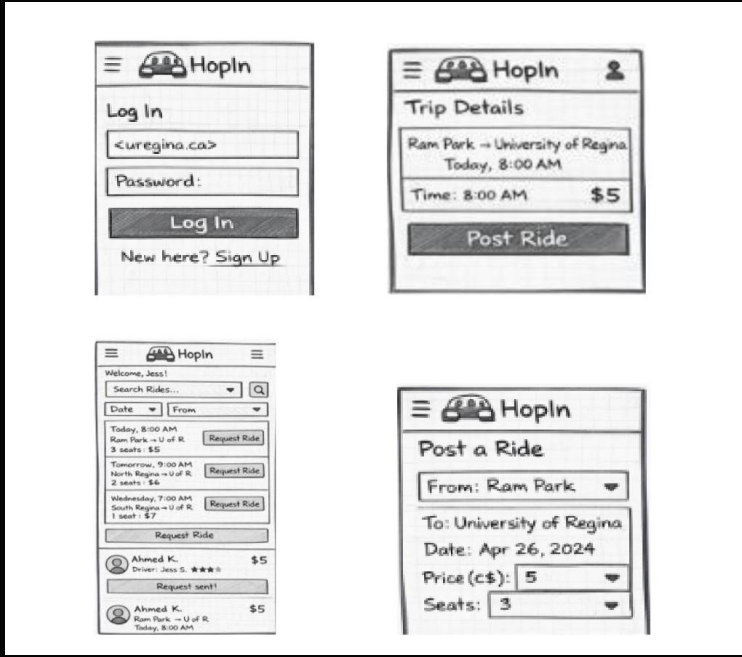
Functional Requirements

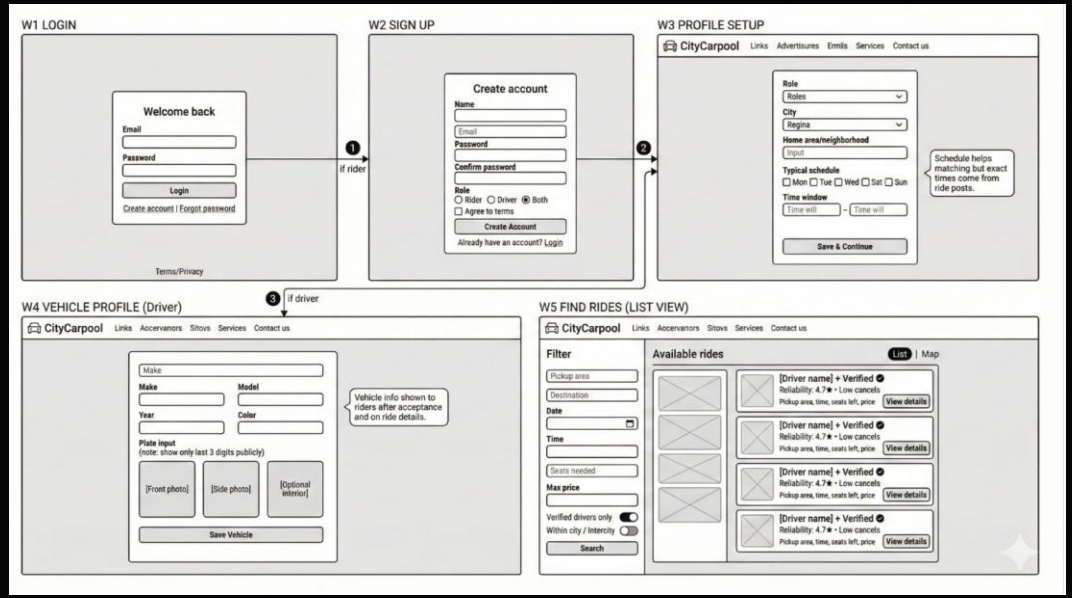
Project Planning Methods



Utilizing Kanban, direct communication, and meeting minutes to track progress

Front End Design



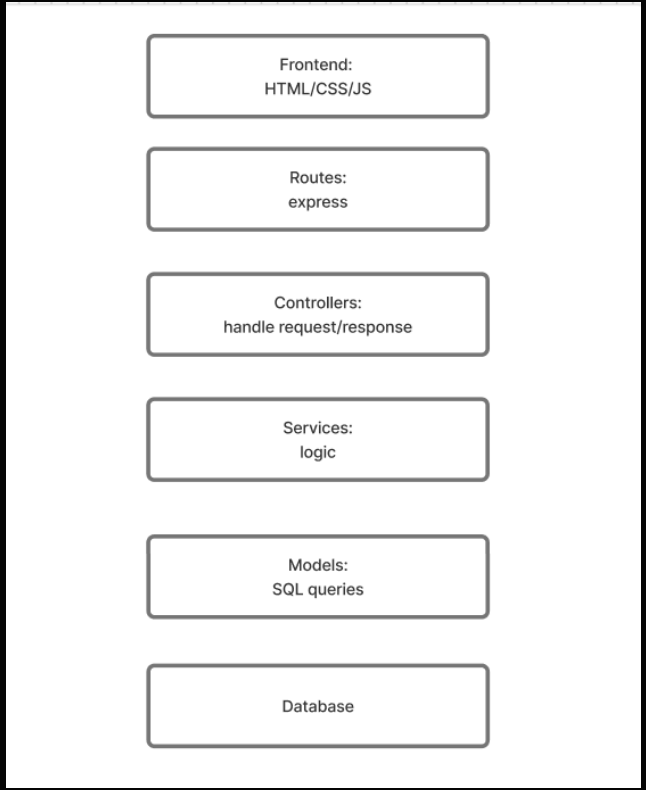


Lo-fidelity Prototypes
(front end architecture)

Tech Stack and Back End Architecture

Folder Structure

```
/backend
 /controllers
  userController.js
  rideController.js
  rideRequestController.js
  ratingController.js
  scheduleController.js
  messageController.js

 /services
  userService.js
  rideService.js
  rideRequestService.js
  ratingService.js
  scheduleService.js
  messageService.js

 /models
  userModel.js
  rideModel.js
  rideRequestModel.js
  ratingModel.js
  scheduleModel.js
  messageModel.js

 /routes
  userRoutes.js
  rideRoutes.js
  rideRequestRoutes.js
  ratingRoutes.js
  scheduleRoutes.js
  messageRoutes.js

 /config
  db.js

 app.js
```



API Endpoints (HTTP protocol)

User

POST /api/users/register
POST /api/users/login
PUT /api/users/:id
GET /api/users/:id

Rides

GET /api/rides
GET /api/rides?location=Downtown
GET /api/rides?time=08:00

POST /api/rides
GET /api/rides/:id
PUT /api/rides/:id
DELETE /api/rides/:id

Ride Requests

POST /api/ride-requests

GET /api/ride-requests?rideId=1
GET /api/ride-requests?userId=2

PUT /api/ride-requests/:id/approve
PUT /api/ride-requests/:id/decline

Ratings

POST /api/ratings
GET /api/users/:id/ratings

Schedule

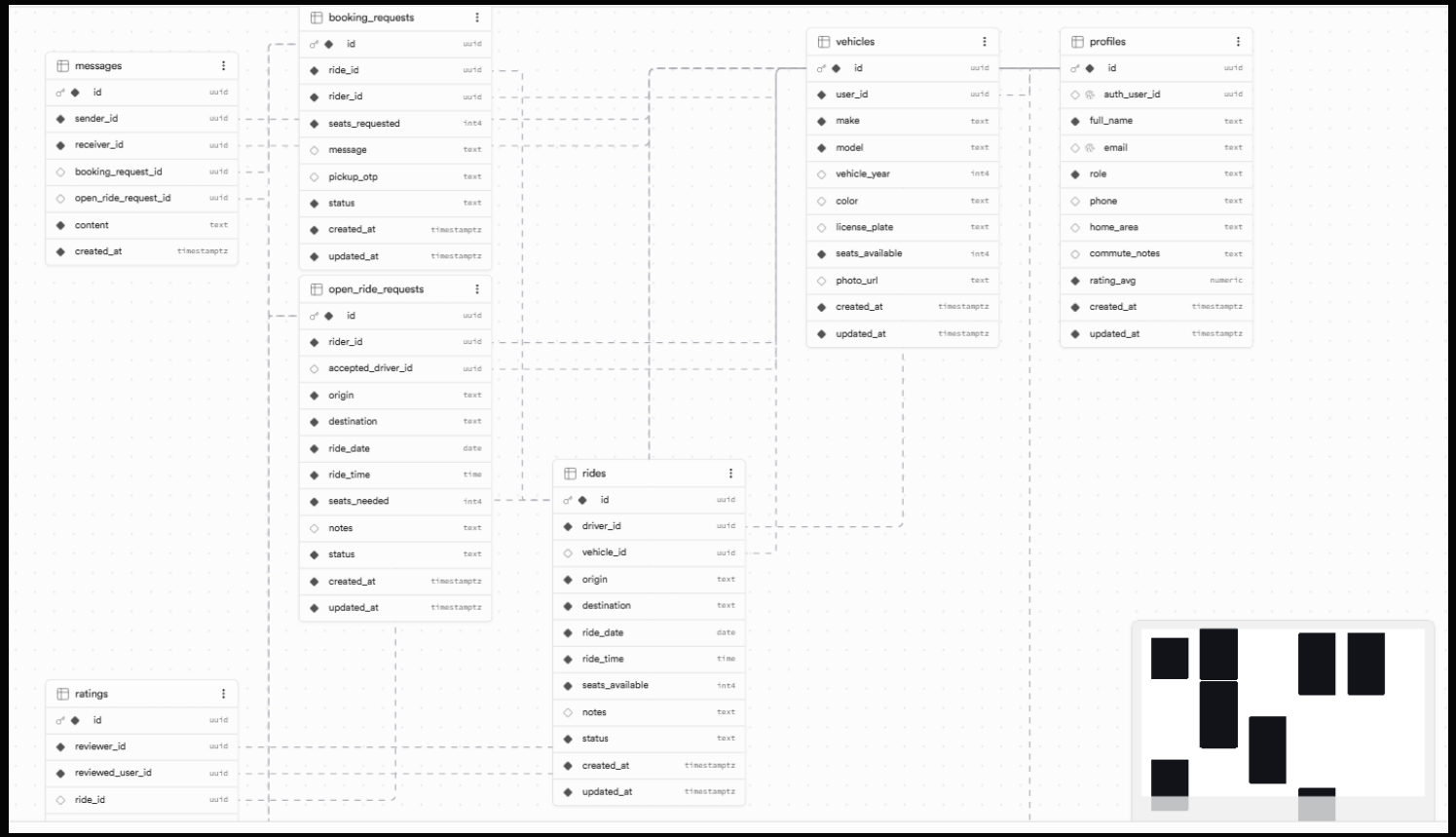
POST /api/schedules
GET /api/schedules/:userId
PUT /api/schedules/:id
DELETE /api/schedules/:id

Messages

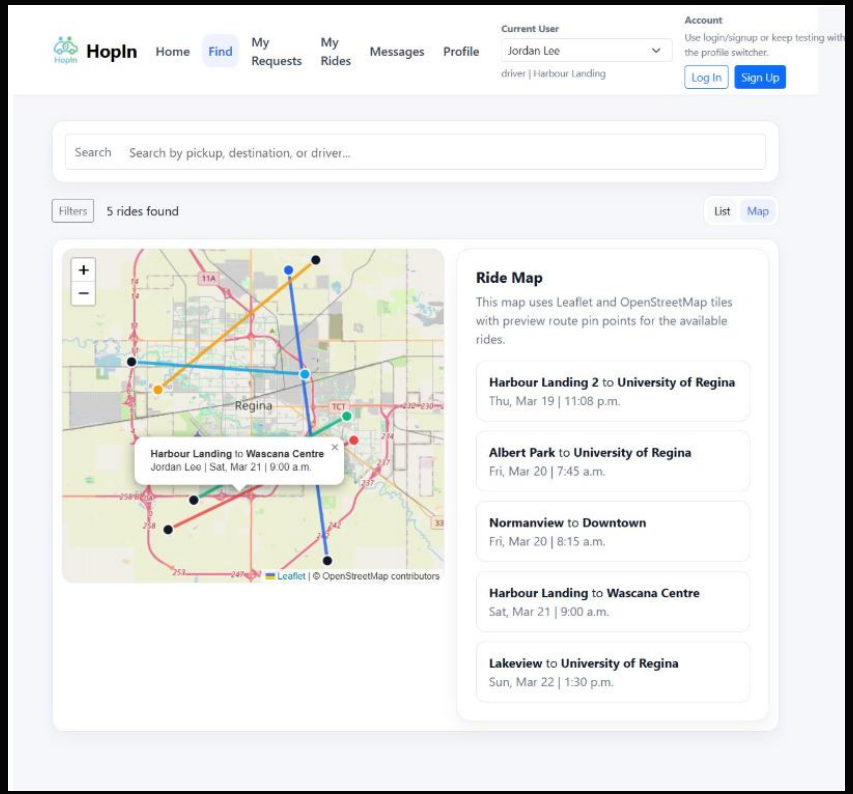
POST /api/messages

GET /api/messages?rideId=1
GET /api/messages?userId=2

Back End Architecture



Future MVP



- GPS tracking of the drivers
- Integrated external payment system
- Drivers license authentication
- 24/7 Chat Support
- Data protection measures (more abstraction and embedded processes)
- Expand to other universities

THANK YOU!

Questions?

Or

Feedbacks